`get_hazard_pointer_for_current_thread()` that returns a reference to your hazard pointer. You then need to set it when you read a pointer that you intend to dereference—in this case the `head` value from the list:

```
std::shared_ptr<T> pop()
{
    std::atomic<void*>& hp=get_hazard_pointer_for_current_thread();
    node* old_head=head.load();    <-1
    node* temp;
    do
    {
        temp=old_head;
        hp.store(old_head);        <-2
        old_head=head.load();
    } while(old_head!=temp);       <-3
    // ...
}
```

You have to do this in a `while` loop to ensure that the `node` hasn't been deleted between the reading of the old `head` pointer ❶ and the setting of the hazard pointer ❷. During this window no other thread knows you're accessing this particular node. Fortunately, if the old `head` node is going to be deleted, `head` itself must have changed, so you can check this and keep looping until you know that the `head` pointer still has the same value you set your hazard pointer to ❸. Using hazard pointers like this relies on the fact that it's safe to use the value of a pointer after the object it references has been deleted. This is technically undefined behavior if you are using the default implementation of `new` and `delete`, so either you need to ensure that your implementation permits it, or you need to use a custom allocator that permits such usage.

Now that you've set your hazard pointer, you can proceed with the rest of `pop()`, safe in the knowledge that no other thread will delete the nodes from under you. Well, almost: every time you reload `old_head`, you need to update the hazard pointer before you dereference the freshly read pointer value. Once you've extracted a node from the list, you can clear your hazard pointer. If there are no other hazard pointers referencing your node, you can safely delete it; otherwise, you have to add it to a list of nodes to be